

RV32IM 5-Stage Pipelined RISC-V Processor

Design and Implementation using Verilog HDL

Project Report by

Nishit Satapara

Department of Electronics and Computer Engineering
Final Year Engineering Project

Abstract

This report documents the design and implementation of a five-stage pipelined RV32IM processor built entirely in Verilog HDL. The motivation behind the project was straightforward: a single-cycle processor is easy to reason about but wastes most of its clock period on instructions that do not need it, and it does not expose any of the real engineering problems that show up the moment instructions start overlapping in time. Building a pipelined core, even a simple one, forces decisions around data hazards, control hazards, and forwarding that a single-cycle design never has to confront.

The processor is organized around eighteen Verilog modules under a single top-level module, `RISC_V_PIPELINED_TOP`. Each pipeline stage — Fetch, Decode, Execute, Memory, and Writeback — is built as its own sub-module containing the functional blocks relevant to that stage, with pipeline registers carrying instruction state forward on every clock edge. A dedicated `Hazard_Unit` sits outside the main datapath and observes register addresses across the Decode, Execute, Memory, and Writeback stages to detect dependencies and generate the forwarding and stalling signals the datapath needs to stay correct.

Data forwarding is implemented from both the Memory stage and the Writeback stage into the Execute stage, with Memory-stage data taking priority since it represents the more recent result. A second, smaller forwarding path operates inside the Decode stage to resolve branch comparisons against values that have not yet been written back to the register file. The design supports the core RV32IM instruction categories — R-type, I-type, load, store, branch, jump, and immediate-format instructions — and was verified through functional simulation using representative instruction sequences covering `ADD`, `SUB`, `ADDI`, `LW`, `SW`, `BEQ`, and `JAL`, with particular attention given to back-to-back dependent instructions where the hazard logic is actually exercised. The outcome is a working, modular pipelined core that demonstrates in hardware, rather than only on paper, why pipelining is inseparable from hazard handling.

1. Introduction

1.1 What a Processor Actually Does

Stripped down to its basics, a processor is a circuit that repeatedly does three things: it reads an instruction from memory, figures out what that instruction means, and carries out the operation the instruction encodes — whether that is an arithmetic computation, a memory access, or a change in control flow. Every architectural decision in a processor design, from how many pipeline stages to use to how hazards are resolved, ultimately exists to make this fetch-decode-execute cycle either faster, more correct, or both. This project is built around that same loop, just split across five overlapping stages instead of one long one.

1.2 RISC as a Design Philosophy

RISC (Reduced Instruction Set Computer) architectures are built on the idea that a small number of simple, fixed-length instructions, each doing one well-defined thing, is easier to pipeline and easier to implement correctly than a large set of variable-length, multi-step instructions. Memory is only ever touched through explicit load and store instructions; every other instruction operates on registers. This load-store discipline turns out to matter a great deal for pipelining specifically, because it keeps the Memory stage simple and isolated — only two instruction types ever need it, and every other instruction passes through that stage doing nothing.

1.3 Why RISC-V

RISC-V is an open instruction set architecture built around a small mandatory base (the integer instruction set) with optional extensions layered on top for multiplication, atomics, floating point, and compressed instructions. For a project like this one, the appeal of RISC-V is that the base instruction encoding is genuinely simple and the specification is freely available, which means every control signal and every opcode decision in this design traces back to a documented encoding rather than to guesswork or to a black-box reference core.

1.4 Why RV32IM Specifically

This project targets RV32IM: the 32-bit base integer instruction set (RV32I) combined with the M extension for integer multiplication and division. RV32I alone is enough to build a processor that pipelines and forwards correctly, but it cannot run anything computationally interesting without resorting to long software loops for multiplication. Adding the M extension gives the processor a complete enough instruction set to execute realistic arithmetic-heavy code while staying well clear of the added complexity of floating-point datapaths or the variable-length decoding that the compressed (C) extension would introduce. RV32IM was, in short, the smallest instruction set that still felt like a real processor rather than a teaching toy.

1.5 Why This Matters as a Project

Most of computer engineering coursework treats the processor as an abstraction sitting underneath the software stack. Designing one removes that abstraction. Decisions that look obvious in a textbook diagram — where to place a pipeline register, which stage should resolve a branch, where forwarding data should originate — stop being obvious the moment they have to

be written as synthesizable Verilog and simulated cycle by cycle. This project was as much about developing that judgment as it was about producing a working processor.

2. Problem Statement

A single-cycle processor executes one complete instruction per clock cycle, which sounds efficient until the clock period itself is examined. Because every instruction must finish in one cycle, the clock period has to be set long enough for the slowest instruction in the entire instruction set — almost always a load, which has to pass through instruction fetch, register read, address calculation, and a memory access before the cycle can end. Every other instruction, including a simple register-to-register ADD, is forced to sit inside that same long cycle even though it finishes its real work far sooner. The clock period is dictated by the worst case, but paid by every instruction.

Pipelining addresses this by splitting instruction execution into stages and letting multiple instructions occupy different stages simultaneously. Instead of one instruction completing every (long) cycle, one instruction completes every (short) cycle once the pipeline is full, because Fetch, Decode, Execute, Memory, and Writeback are now five independent pieces of hardware working on five different instructions at once. This is the entire reason pipelining is worth doing — it raises instruction throughput without requiring faster transistors.

The problem is that overlapping instructions breaks an assumption the single-cycle design never had to think about: that an instruction always sees fully up-to-date register and memory state when it executes. In a pipeline, an instruction in the Execute stage may need a register value that an earlier instruction, still sitting in the Memory or Writeback stage, has not yet written back. Read that value directly from the register file and the Execute stage gets stale data — silently, with no indication anything went wrong. Branches create a related but distinct problem: by the time a branch is resolved in the Execute stage, the two or three instructions immediately following it in program order have already been fetched and have started moving down the pipeline, regardless of whether the branch was taken. Both problems — data hazards and control hazards — are not corner cases. They show up constantly in ordinary code, and a pipelined processor that does not handle them is not faster, it is simply wrong.

3. Project Objectives

- Design and implement a 32-bit RV32IM processor datapath capable of executing the core RISC-V integer instruction categories.
- Restructure the datapath into a five-stage pipeline (Fetch, Decode, Execute, Memory, Writeback) with clocked pipeline registers between every stage.
- Implement a dedicated hazard detection and forwarding unit that resolves data hazards without relying on software-inserted NOPs.
- Handle control hazards arising from branch instructions by resolving branch outcome and target address in the Execute stage and flushing incorrectly fetched instructions.
- Implement the full design in synthesizable Verilog HDL using a modular, stage-wise hierarchy rather than one monolithic datapath block.

- Functionally verify the processor through simulation, using instruction sequences that specifically exercise forwarding, stalling, and branch redirection rather than only independent instructions.

4. System Architecture

The processor is built top-down, starting from a single top-level module and decomposing it stage by stage rather than building one large datapath and cutting it into pieces afterward. At the top sits `RISC_V_PIPELINED_TOP`, which instantiates the five stage modules — `Fetch_Cycle`, `Decode_Cycle`, `Execute_Cycle`, `Memory_Cycle`, and `Writeback_Cycle` — plus the `Hazard_Unit`, and wires the pipeline registers between them. Each stage module is, in effect, a self-contained mini-datapath that only knows about the signals relevant to its own stage; it does not need to know what happens two stages later. This separation made the hazard logic the only place in the design where signals from non-adjacent stages had to be compared directly, which kept the rest of the datapath easy to reason about in isolation.

Within each stage module, the architecture continues to follow the same top-down breakdown: major functional modules handle the actual computation (`ALU`, `Register_File`, `Control_Unit_1`, `Branch_Module`, `Data_Memory`, `Instruction_Memory`), while supporting modules handle data movement and selection (`PC_Module`, `Sign_Extend`, and the `Mux_2_1_32` / `Mux_4_1_32` multiplexers reused across multiple stages wherever a value needs to be chosen from several possible sources).

Instructions move through the processor in program order, one stage per clock cycle, with up to five instructions in flight simultaneously once the pipeline is full:

Instruction Fetch	Instruction Decode	Execute	Memory Access	Write Back
-------------------	--------------------	---------	---------------	------------

An instruction fetched in cycle N reaches Decode in cycle $N+1$, Execute in cycle $N+2$, Memory in cycle $N+3$, and Writeback in cycle $N+4$, while the next instruction in program order is one cycle behind it at every point. The `Hazard_Unit` continuously monitors the register addresses moving through this pipeline and intervenes — by forwarding a value early or by stalling a stage — whenever two instructions in flight at the same time depend on each other.

5. Pipeline Design

The five stages below are described in the order an instruction actually passes through them. Each description follows the same pattern: why the stage has to exist at all, what it does, how it is built, and what it hands off to the stage after it.

5.1 Fetch Stage

Why it exists: Every instruction has to be retrieved from memory before anything else in the processor can act on it, and the program counter has to be kept correct in the presence of branches that redirect execution.

What it does: Maintains the program counter, reads the instruction word at that address from Instruction_Memory, and computes the next PC value — either the sequential PC+4 or a branch target supplied by the Execute stage.

How it works: PC_Module is a 32-bit register that updates on every rising clock edge with the value selected by a Mux_2_1_32. The multiplexer chooses between PC+4 (the default) and PCTargetE, the branch target computed in Execute, based on the PCSrcE control signal. Instruction_Memory is addressed combinationally by the current PC and returns the 32-bit instruction word with no clock delay of its own.

Inputs: PCSrcE and PCTargetE, fed back from the Execute stage.

Outputs: PC (current address) and Instr (fetched instruction word), latched into the Fetch/Decode pipeline register along with PC+4 for later use by jump instructions.

Interaction with the next stage: The IF/ID pipeline register passes the raw instruction word and PC+4 forward into Decode, where the instruction is actually interpreted for the first time.

5.2 Decode Stage

Why it exists: An instruction is just a 32-bit pattern until its fields are extracted and turned into control signals and register values; the datapath cannot act on raw bits.

What it does: Splits the instruction into its opcode, function code, and register address fields; generates the control word for the rest of the pipeline; reads the two source register values; and builds the sign-extended immediate where the instruction format calls for one.

How it works: Control_Unit_1 decodes the opcode (and funct3/funct7 fields where the opcode alone is ambiguous, such as distinguishing ADD from SUB) into the RegWrite, ALUSrc, MemWrite, Jump, Branch, ResultSrc, and ALUControl signals that govern every later stage. Register_File is read combinationally on the RS1 and RS2 fields, returning RD1 and RD2 in the same cycle. Sign_Extend inspects the instruction format and produces a correctly sign- or zero-extended 32-bit immediate, since I-type, S-type, B-type, and J-type instructions all place their immediate bits in different positions within the word.

Inputs: Instr and PC+4 from the IF/ID pipeline register.

Outputs: RD1, RD2, the sign-extended immediate, the full control word, and the RS1/RS2/RD register addresses (needed downstream by the Hazard_Unit), latched into the Decode/Execute pipeline register.

Interaction with the next stage: The ID/EX pipeline register passes operand values and control signals into Execute, while the register addresses it carries are also tapped by the Hazard_Unit to detect dependencies before the next instruction even reaches Execute.

5.3 Execute Stage

Why it exists: This is the stage where actual computation happens and where branch outcomes are resolved, which makes it the stage most exposed to data hazards — it is the first point in the pipeline where stale operand data would actually produce a wrong result.

What it does: Performs the arithmetic or logical operation selected by ALUControl, evaluates the branch condition and computes the branch target address when the instruction is a branch, and

selects the correct ALU operands by applying forwarding where the Hazard_Unit has detected a dependency.

How it works: The ALU takes its first operand from the (possibly forwarded) RD1 value and its second operand from either RD2 or the sign-extended immediate, selected by a Mux_2_1_32 driven by ALUSrc. Ahead of the ALU itself, the true operand values are chosen by forwarding multiplexers driven by ForwardAE and ForwardBE, which select between the raw Register_File value, the Memory-stage ALU result (ALU_ResultM), and the Writeback-stage result (ResultW). Branch_Module compares the two operand values for branch instructions and produces PCSrcE (whether the branch is taken) together with PCTargetE (PC plus the branch immediate), both of which are sent directly back to the Fetch stage rather than waiting for later pipeline registers, since redirecting the PC as early as possible limits how many wrongly fetched instructions need to be discarded.

Inputs: RD1, RD2, the immediate, control signals from ID/EX, and ForwardAE/ForwardBE from the Hazard_Unit.

Outputs: ALU_ResultE, WriteDataE (the value to be stored for a SW instruction), PCTargetE, and PCSrcE.

Interaction with the next stage: The EX/MEM pipeline register carries the ALU result and store data into the Memory stage, while PCSrcE and PCTargetE are routed immediately back to Fetch to redirect the PC on a taken branch.

5.4 Memory Stage

Why it exists: Load and store instructions are the only ones that need to touch data memory, and isolating that access to its own stage keeps every other instruction's timing unaffected by memory latency.

What it does: Reads from or writes to Data_Memory using the address computed by the ALU in the previous stage.

How it works: Data_Memory is addressed by ALU_ResultM. When MemWriteM is asserted (a store instruction), WriteDataM is written to that address; for a load instruction, the word at that address is returned as ReadDataM. Every instruction that is neither a load nor a store passes through this stage with Data_Memory simply idle.

Inputs: ALU_ResultM, WriteDataM, and the MemWriteM control bit from the EX/MEM pipeline register.

Outputs: ReadDataM (for loads) and ALU_ResultM, passed through to Writeback.

Interaction with the next stage: The MEM/WB pipeline register carries the memory read result and the ALU result forward into Writeback. ALU_ResultM is also tapped directly by the Execute stage's forwarding multiplexers, since an instruction sitting in Memory is frequently the source of a dependency for the instruction currently in Execute.

5.5 Writeback Stage

Why it exists: By the time an instruction reaches this stage it may carry a result from three different places — the ALU, data memory, or PC+4 for a JAL — and the register file needs one consistent value to write, regardless of which kind of instruction produced it.

What it does: Selects the correct result source and writes it back into the register file.

How it works: A Mux_4_1_32 selects between ALU_ResultW, ReadDataW, and PCPlus4W based on the ResultSrcW control signal generated back in Decode, producing the final ResultW value. ResultW is written into Register_File at address RDW whenever RegWriteW is asserted.

Inputs: ALU_ResultW, ReadDataW, PCPlus4W, ResultSrcW, RegWriteW, and RDW from the MEM/WB pipeline register.

Outputs: ResultW, written into the register file.

Interaction with the next stage: There is no stage after Writeback, but ResultW does not simply disappear into the register file — it is also fed back as a forwarding source into the Execute stage (and into the Decode-stage forwarding paths for branch comparisons), since an instruction still in Execute may depend on a result that has only just finished Writeback.

6. Module Overview

The processor is built from eighteen Verilog modules. Rather than repeat the same why / what / how / result pattern eighteen times, the table below gives a quick reference for every module, and the paragraphs that follow it go into more depth on the modules that carry the most architectural weight — the ones an interviewer is most likely to ask about.

Module	Role	Function
RISC_V_PIPELINED_TOP	Top-level integration	Instantiates all five stage modules and the Hazard_Unit; wires pipeline registers and feedback paths between them.
Fetch_Cycle	Stage wrapper	Houses PC_Module, Instruction_Memory, and the PC-select Mux_2_1_32.
Decode_Cycle	Stage wrapper	Houses Control_Unit_1, Register_File, Sign_Extend, and a Mux_2_1_32 used in operand selection.
Execute_Cycle	Stage wrapper	Houses the ALU, Branch_Module, the ALUSrc Mux_2_1_32, and the forwarding Mux_4_1_32 blocks.
Memory_Cycle	Stage wrapper	Houses Data_Memory and the load/store control logic for that stage.
Writeback_Cycle	Stage wrapper	Houses the result-select Mux_4_1_32 that produces ResultW.
Hazard_Unit	Dependency resolution	Compares register addresses across Decode, Execute, Memory, and Writeback; generates ForwardAE/ForwardBE, ForwardAD/ForwardBD, and stall/flush signals.
Register_File	Operand storage	32 x 32-bit register array, two combinational read ports, one clocked write port.
ALU	Computation	Performs ADD, SUB, AND, OR, XOR, SLT, shifts, and address calculations; produces Zero, Carry, Overflow, and Negative flags.

Module	Role	Function
Control_Unit_1	Instruction decode	Maps opcode / funct3 / funct7 fields to RegWrite, ALUSrc, MemWrite, Jump, Branch, ResultSrc, and ALUControl.
Instruction_Memory	Program storage	Combinational ROM-style read, addressed by PC, returns the 32-bit instruction word.
Data_Memory	Data storage	Synchronous write / combinational read RAM, addressed by the ALU result, used by load and store instructions.
Branch_Module	Branch resolution	Evaluates the branch condition and computes PCTargetE; produces PCSrcE.
Sign_Extend	Immediate generation	Builds the correctly sign/zero-extended 32-bit immediate for I, S, B, and J formats.
PC_Module	Program counter	Clocked 32-bit register holding the current fetch address.
Mux_2_1_32	Selection	Two-input, 32-bit multiplexer reused for PC selection, ALU source selection, and other binary choices across stages.
Mux_4_1_32	Selection	Four-input, 32-bit multiplexer used for Execute-stage forwarding and for the Writeback result selection.

6.1 Register_File

The register file exists because every RISC-V instruction other than loads and stores reads and writes registers rather than memory, so its read latency effectively sets a floor under how fast Decode can run. It is built as a 32-by-32-bit array with two independent, purely combinational read ports addressed by RS1 and RS2, and a single write port that commits on the clock edge. Reads and the write happen in the same module so that a write occurring in the same cycle as a read to the same address can be handled deliberately (write-then-read ordering) rather than left to chance. In practice this module is small, but getting its read/write timing right matters more than its size suggests, since the Hazard_Unit's entire forwarding strategy assumes the register file's behavior on simultaneous read/write to the same address is well defined.

6.2 ALU

The ALU is the only module instantiated solely for computation rather than for moving data around, and it has to support every R-type and I-type arithmetic and logic operation in RV32I — addition, subtraction, AND, OR, XOR, set-less-than, and the shift operations — plus the effective-address addition that loads and stores rely on. ALUControl, generated back in Decode, selects the operation, and the ALU produces Zero, Carry, Overflow, and Negative flags alongside the result, with Zero specifically consumed by Branch_Module for equality-based branches such as BEQ. Folding address calculation into the same ALU rather than building a separate adder kept the Execute stage's datapath simpler, since loads, stores, and ordinary arithmetic instructions all

flow through the same hardware path differing only in the ALUControl value and the source operand selected by ALUSrc.

6.3 Control_Unit_1

Control_Unit_1 is what turns a static instruction encoding into the dynamic behavior the rest of the datapath depends on. It looks at the opcode field, and for instructions where the opcode alone does not fully specify the operation (R-type instructions in particular, where ADD and SUB share an opcode and differ only in funct7), it also inspects funct3 and funct7 to produce the correct ALUControl value. The output control word — RegWrite, ALUSrc, MemWrite, Jump, Branch, ResultSrc, ALUControl — is generated entirely in Decode and then carried down the pipeline alongside the data it governs, so that by the time an instruction reaches, say, the Memory stage, the MemWrite bit it carries is the one that was decoded for that exact instruction rather than something recomputed late and prone to becoming associated with the wrong instruction.

6.4 Branch_Module

Branch_Module exists because a branch decision needs more than just an ALU comparison — it needs that comparison combined with the Branch control bit, and it needs a target address computed independently of the ALU, which is busy evaluating the comparison itself. It produces PCSrcE by combining the Branch control signal with the Zero flag (for BEQ-style equality branches), and computes PCTargetE as the sum of the branch instruction's own PC and its sign-extended immediate. Both outputs leave the Execute stage immediately rather than waiting for a pipeline register, because every cycle a taken branch's redirection is delayed is another wrongly fetched instruction that has to be discarded.

6.5 Mux_2_1_32 and Mux_4_1_32

Rather than write a custom multiplexer at every point in the datapath where a value has to be chosen between alternatives, the design reuses two generic, parameterized multiplexer modules. Mux_2_1_32 is instantiated wherever a binary choice is needed — PC versus branch target in Fetch, register value versus immediate in Execute's ALUSrc path — and Mux_4_1_32 is used wherever up to four sources compete, most importantly in the Execute-stage forwarding logic, where an operand might come from the register file, from the Memory stage, or from the Writeback stage, and in the Writeback stage itself, where the final result might be an ALU output, a memory read, or PC+4. Reusing the same two modules everywhere a selection is needed kept the datapath's structure visually consistent and made the forwarding multiplexers easy to recognize as forwarding logic rather than as one-off custom circuits.

7. Hazard Handling

Pipelining only pays off if the pipeline produces the same results a single-cycle processor would have produced for the same program, just faster. The moment instructions overlap, three categories of hazard can break that guarantee, and the Hazard_Unit exists specifically to detect and resolve them before they reach a stage where they would corrupt a result.

7.1 Data Hazards

A data hazard occurs when an instruction needs a register value that an earlier, still-in-flight instruction has not yet written back. Consider ADD x5, x1, x2 immediately followed by SUB x6, x5, x3. By the time the SUB instruction reaches Execute, the ADD instruction that produced x5 is only in the Memory stage — its result has not yet reached the register file, and a direct read of Register_File would return the old value of x5. Left unhandled, this would silently produce an incorrect result for SUB.

7.2 Control Hazards

A control hazard occurs because the processor does not know whether a branch is taken until that branch reaches Execute, by which point the instructions immediately after it in program order have already been fetched into the pipeline. If the branch turns out to be taken, those fetched instructions were never supposed to execute and have to be discarded (flushed) before they can write back any results or trigger a memory access.

7.3 Structural Hazards

A structural hazard occurs when two instructions need the same piece of hardware in the same cycle. Keeping Instruction_Memory and Data_Memory as two physically separate modules, rather than one shared memory, avoids the most common structural hazard a simple pipeline would otherwise face — an instruction fetch and a data access competing for the same memory port in the same cycle. This is a structural decision rather than a piece of logic, but it is deliberate: it removes an entire category of hazard at the architecture level instead of needing to detect and stall around it at runtime.

7.4 Forwarding

Forwarding is the primary mechanism this design uses to resolve data hazards without stalling. Instead of waiting for a result to be written back to the register file and then read out again, the value is routed directly from the pipeline stage that produced it to the pipeline stage that needs it, several cycles earlier than a register file round-trip would allow.

Execute-stage forwarding: ForwardAE and ForwardBE select the true ALU operands from among three sources: the raw Register_File output, ALU_ResultM (the result currently sitting in the Memory stage), and ResultW (the result currently being written back). Because an instruction in the Memory stage is more recent than one in Writeback, Memory-stage forwarding takes priority over Writeback-stage forwarding whenever both could apply to the same register.

Decode-stage forwarding: ForwardAD and ForwardBD resolve a narrower but equally important problem: branch instructions evaluate their comparison using values read in Decode, one stage earlier than the Execute-stage forwarding paths operate. Without a separate forwarding path here, a branch immediately following an instruction that has not yet written back its result would compare against a stale value and could resolve in the wrong direction.

Working through the ADD / SUB example again with forwarding in place: when SUB reaches Execute, the Hazard_Unit compares RD_M (the destination register the Memory-stage instruction is about to write) against RS1_E and RS2_E (the source registers SUB needs in Execute). The match on x5 sets ForwardAE to select ALU_ResultM instead of the register file output, and SUB computes against the correct, forwarded value of x5 — one cycle earlier than the value would have been available through the register file.

7.5 Stalling

Forwarding resolves most data hazards, but not all of them. A load instruction does not have its data available until the end of the Memory stage, so an instruction immediately following a load that depends on the loaded value (a load-use hazard) cannot be resolved by forwarding alone — the data simply does not exist yet at the point Execute would need it. For this specific case, the pipeline has to stall: the dependent instruction is held in Decode for one extra cycle (and a bubble is inserted into Execute in its place) until the load's result becomes available in Memory and can be forwarded normally on the following cycle.

7.6 Hazard Detection Unit

The Hazard_Unit's detection logic works by continuously comparing the destination register addresses of instructions further along the pipeline (RD_M in Memory, RD_W in Writeback) against the source register addresses of instructions earlier in the pipeline (RS1_D, RS2_D in Decode and RS1_E, RS2_E in Execute). Whenever a destination address matches a source address — and the corresponding RegWrite signal confirms a write is actually going to happen, since address zero and non-register-writing instructions should never trigger forwarding — the unit asserts the corresponding ForwardAE, ForwardBE, ForwardAD, or ForwardBD signal. The load-use case is detected separately, by checking whether the instruction currently in Execute is a load and whether its destination register matches a source register of the instruction currently in Decode, in which case the stall and bubble-insertion logic takes over instead of a forwarding signal.

8. Simulation and Verification

Verification was carried out through functional simulation of the completed datapath, using instruction sequences written directly into Instruction_Memory and observing register and memory state as instructions moved through the pipeline cycle by cycle. The intent was not simply to confirm that isolated instructions produced correct results, since a single-cycle processor would pass that test just as easily — the sequences specifically targeted the situations where pipelining changes behavior: back-to-back dependent instructions, loads immediately followed by their consumers, and branches surrounded by instructions that should or should not end up executing.

8.1 Arithmetic Instructions — ADD, SUB, ADDI

Independent ADD and SUB instructions were used first to confirm the baseline datapath — instruction decode, register read, ALU operation, and writeback — was correct with no hazards in play. Dependent sequences (an ADD followed immediately by a SUB consuming its result) were then used specifically to exercise Execute-stage forwarding, with the simulation confirming that ForwardAE or ForwardBE asserted correctly on the cycle the dependent instruction reached Execute, and that the ALU received the forwarded value rather than the stale register file output. ADDI sequences confirmed the Sign_Extend and ALUSrc-selected immediate path independently of register-to-register operations.

8.2 Memory Instructions — LW, SW

SW instructions were used to confirm that ALU_ResultM correctly carried the computed effective address into the Memory stage and that WriteDataM was written to Data_Memory only when

MemWriteM was asserted. LW instructions confirmed the reverse path, with ReadDataM correctly reaching the Writeback stage and being selected by ResultSrcW. A load immediately followed by a dependent instruction was used specifically to confirm the load-use stall: the simulation showed the dependent instruction held for one additional cycle in Decode, a bubble propagating into Execute in its place, and the dependent instruction then executing correctly once the loaded value became available for forwarding.

8.3 Branch Instructions — BEQ

BEQ sequences were tested for both outcomes. For the not-taken case, the simulation confirmed PCSrcE remained deasserted and the pipeline continued fetching sequentially with no disruption. For the taken case, PCSrcE asserted in the cycle BEQ reached Execute, PCTargetE was confirmed to equal the branch's own PC plus its immediate, and the instructions that had already been fetched into the pipeline behind the branch were flushed rather than allowed to write back results. A BEQ immediately preceded by an instruction writing one of the compared registers was used to confirm the Decode-stage forwarding paths (ForwardAD/ForwardBD), since this is the one case where the branch's correctness depends on forwarding happening a stage earlier than the Execute-stage paths operate.

8.4 Jump Instructions — JAL

JAL was tested to confirm that PCPlus4W was correctly selected as the writeback result (since JAL's defining behavior is storing the return address rather than an ALU output), and that the jump itself correctly redirected the PC without waiting on the Branch_Module's comparison logic, which is specific to conditional branches.

8.5 Outcome

Across these scenarios, the pipeline produced results consistent with what the same instruction sequences would have produced on a non-pipelined datapath, while taking visibly fewer cycles overall once the pipeline was full. The cases that mattered most for confidence in the design were not the independent instructions — those would pass on almost any datapath — but the dependent sequences, the load-use stall, and the branch flush, since those are precisely the behaviors that distinguish a correct pipelined processor from one that merely looks like one.

9. Applications and Future Scope

9.1 Applications

- Embedded systems, where a small, well-understood instruction set and predictable timing matter more than raw performance.
- IoT devices, where a compact RV32IM core can serve as the control processor for sensor and connectivity logic.
- Educational processors, since the modular stage-wise structure of this design makes it straightforward to trace a single instruction through the entire pipeline by hand.
- As a starting point for custom RISC-V cores, where the base datapath can be extended with additional instructions or peripherals without restructuring the existing pipeline.

9.2 Future Scope

- Cache integration ahead of Instruction_Memory and Data_Memory, to move the design closer to a realistic memory hierarchy rather than the flat memories used here.
- Branch prediction, to recover the cycles currently lost to flushing on every taken branch, rather than always assuming branches are not taken.
- FPGA deployment, to validate timing and resource usage on real hardware rather than in simulation alone.
- A full RTL-to-GDS flow, carrying the design through synthesis and physical implementation to evaluate area, timing, and power on a target process.
- General performance optimization, including deeper pipelining or superscalar issue, once the current five-stage design is fully characterized.

10. Conclusion

This project set out to build a working five-stage pipelined RV32IM processor, and the result is a modular Verilog design where each pipeline stage, the hazard-handling logic, and the supporting datapath modules can all be examined and tested independently of one another. The most significant outcome of the project was not the final processor itself but the understanding built along the way: pipelining is conceptually simple to describe but genuinely hard to get right, because almost every correctness problem in a pipelined processor traces back to instructions overlapping in ways a single-cycle design never had to consider.

Implementing the Hazard_Unit in particular made clear why forwarding and stalling are not optional refinements but a core part of what makes a pipelined processor correct in the first place. A pipeline without hazard handling is not simply slower or less efficient than one with it — it produces wrong answers on perfectly ordinary code, the moment one instruction depends on the result of the instruction just before it. Resolving that, along with the related problem of discarding wrongly fetched instructions after a taken branch, was where most of the real engineering effort in this project went, and it is also where the project's value as preparation for further work in computer architecture mainly lies.